

BÁO CÁO THỰC HIỆN XÁC THỰC VÀ PHÂN QUYỀN

Dự án E-Healthcare Portal

Đồng Văn Tú

15/09/2026

Thông tin	Nội dung
Phạm vi	Đăng ký, OTP, PHR, đăng nhập, Google OAuth, JWT và RBAC
Công nghệ mục tiêu	NestJS 10.x, TypeScript, PostgreSQL 16, TypeORM 0.3.x, bcrypt, JSON Web Token
Nhánh thực hiện	feature/srs-auth-01-registration
Nhánh tích hợp	develop
Ngày báo cáo	15/09/2026
Người thực hiện	Đồng Văn Tú

1. Mục tiêu và phạm vi

Xây dựng mô-đun xác thực cho E-Healthcare Portal theo các yêu cầu SRS-AUTH-01 và SRS-AUTH-02: đăng ký bằng Email hoặc Số điện thoại, xác minh OTP, tạo Hồ sơ Sức khỏe Cá nhân (PHR), đăng nhập bằng mật khẩu, đăng nhập Google, cấp JWT và kiểm soát phân quyền RBAC.

Task	Căn cứ	Yêu cầu chính	Trạng thái
1	SRS-AUTH-01	Kiểm tra mật khẩu mạnh; băm BCrypt cost factor 12	Hoàn thành
2	SRS-AUTH-01; NFR-SEC-03	OTP 6 số, TTL 3 phút; giới hạn gửi	Hoàn thành

Task	Căn cứ	Yêu cầu chính	Trạng thái
		và khóa khi nhập sai	
3	SRS-AUTH-01	Tạo PHR rỗng sau khi đăng ký thành công	Hoàn thành
4	SRS-AUTH-02	Đăng nhập; khóa 30 phút sau 5 lần sai liên tiếp	Hoàn thành
5	SRS-AUTH-02, mục 3.3	Google OAuth 2.0 / OpenID Connect	Hoàn thành ở mức mã nguồn và cấu hình
6	SRS-AUTH-02; NFR-SEC-02	Access Token 15 phút, Refresh Token 7 ngày trong cookie an toàn	Hoàn thành
7	Mục 2.3	RBAC: PATIENT, DOCTOR, RECEPTIONIST, ADMIN	Hoàn thành

2. Phương án triển khai

- Tách trách nhiệm theo service: AuthService xử lý đăng ký/OTP; LoginService xử lý mật khẩu; SessionService quản lý JWT; GoogleAuthService xử lý OAuth; guard xử lý xác thực và RBAC.
- Không lưu mật khẩu hoặc OTP dạng rõ. Mật khẩu dùng BCrypt cost 12; OTP và Refresh Token được lưu dạng hash.
- Các thao tác tạo User, role và PHR nằm trong transaction để tránh dữ liệu tạo dang dở.
- Các request đồng thời theo cùng Email/Số điện thoại được khóa transaction để không làm sai bộ đếm OTP hoặc tạo trùng tài khoản.
- Refresh Token được xoay vòng sau mỗi lần làm mới; token cũ bị dùng lại sẽ thu hồi session.

Đăng ký → gửi OTP → xác minh OTP → User ACTIVE + ROLE_PATIENT + PHR
 Đăng nhập/Google → SessionService → Access Token + Refresh Token
 Request bảo vệ → AccessTokenGuard → RolesGuard → Controller

3. Kết quả thực hiện

3.1. Đăng ký, mật khẩu và OTP

File chính: `server/src/auth/auth.service.ts`

DTO: `server/src/auth/dto/register.dto.ts`

Tiện ích mã hóa: `server/src/common/utils/crypto.util.ts`

Nội dung	Kết quả
Mật khẩu	Tối thiểu 8 ký tự; có chữ hoa, chữ thường, số và ký tự đặc biệt
Băm mật khẩu	bcrypt, cost factor 12; từ chối mật khẩu quá 72 byte
OTP	Sinh ngẫu nhiên 6 chữ số; hiệu lực 180 giây
Gửi lại OTP	Chờ tối thiểu 60 giây
Hạn mức SMS	Tối đa 3 SMS thành công/SĐT/10 phút
Nhập OTP sai	Lưu bộ đếm; sai 5 lần khóa 900 giây (15 phút)
Chống dùng lại	Phiên OTP bị xóa khi xác minh thành công hoặc thay thế khi gửi OTP mới

Endpoint:

<code>POST /api/v1/auth/register/otp</code>	→ yêu cầu OTP
<code>POST /api/v1/auth/register/verify</code>	→ xác minh OTP

3.2. Tạo PHR và dữ liệu đăng ký

Migration: `1789477200000-create-registration-and-phr.ts`

Sau khi OTP hợp lệ, transaction tạo lần lượt:

<code>users (status = ACTIVE)</code>
→ <code>user_roles (ROLE_PATIENT)</code>
→ <code>personal_health_profiles</code> (bản ghi rỗng)
→ xóa <code>registration_sessions</code>

`personal_health_profiles.user_id` là khóa ngoại duy nhất tới `users`, bảo đảm mỗi User có tối đa một PHR và PHR bị xóa theo User.

3.3. Đăng nhập bằng mật khẩu

File: server/src/auth/login.service.ts

Tình huống	Xử lý
Email/SĐT hoặc mật khẩu sai	Trả lỗi chung, không tiết lộ định danh có tồn tại hay không
Sai dưới 5 lần	Tăng failed_login_attempts
Sai lần thứ 5	Đặt login_locked_until = hiện tại + 30 phút; thu hồi session còn hoạt động
Mật khẩu đúng	Xóa bộ đếm sai và cấp session mới

Endpoint: POST /api/v1/auth/login được giới hạn 10 request/phút/client.

3.4. Google OAuth 2.0 / OpenID Connect

File: server/src/auth/google-auth.service.ts

Controller: server/src/auth/google-auth.controller.ts

Bước	Kết quả
Khởi tạo	Tạo state, nonce, browser token và PKCE code verifier/challenge
Callback	Đổi authorization code, xác minh chữ ký ID Token, issuer, audience, hạn dùng, email đã xác minh và nonce
User cũ	Liên kết Google subject hợp lệ và cấp session
User mới	Lưu phiên hoàn tất 10 phút, yêu cầu bổ sung họ tên, giới tính, ngày sinh
Hoàn tất	Tạo User ACTIVE, ROLE_PATIENT, PHR và session

Endpoint:

```
GET /api/v1/auth/google
GET /api/v1/auth/google/callback
POST /api/v1/auth/google/complete
```

3.5. JWT, phiên đăng nhập và RBAC

File: server/src/auth/session.service.ts

Controller: server/src/auth/session.controller.ts

Guard: server/src/auth/auth.guard.ts

Thành phần	Kết quả
Access Token	HS256, chứa userId, role, sid; TTL 900 giây (15 phút)
Refresh Token	Secret riêng, TTL tối đa 604800 giây (7 ngày), hash SHA-256 trong CSDL
Cookie Refresh	HttpOnly, Secure, SameSite=Strict, path /api/v1/auth
Rotation	Refresh thành công cấp token mới và thay hash cũ
Replay	Refresh Token cũ bị dùng lại sẽ thu hồi session
RBAC	ROLE_PATIENT, ROLE_DOCTOR, ROLE_RECEPTIONIST, ROLE_ADMIN
Mặc định an toàn	Route bảo vệ không khai báo @Roles() bị từ chối với 403

Endpoint phiên:

```
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
GET /api/v1/auth/me
```

4. Trạng thái kiểm thử và bằng chứng

Hạng mục	Kết quả	Mức xác nhận
Quy tắc mật khẩu và BCrypt	Pattern kiểm tra đầy đủ 4 nhóm ký tự; cost = 12	Kiểm tra mã nguồn và unit test
OTP/giới hạn request	TTL, resend, giới hạn SMS, sai 5 lần khóa 15 phút	Unit/HTTP test

Hạng mục	Kết quả	Mức xác nhận
Đăng nhập	Đếm lỗi, khóa 30 phút, thu hồi session khi khóa	Unit test
JWT/Refresh Token	Ký, xác minh, rotation, replay revoke, logout	Unit test
RBAC	Access guard, role guard, deny-by-default	Unit test và kiểm tra mã nguồn
Google OAuth	Kiểm tra state, nonce, PKCE và callback trong mã nguồn	Unit test với mock; cần credentials thật để kiểm thử liên kết Google thực tế
Backend	npm test: 54/54 test pass; npm run build: pass	Chạy ngày 15/09/2026

Kết luận kiểm thử: các yêu cầu trong phạm vi checklist đạt ở mức mã nguồn, test tự động backend và build TypeScript.

5. Hướng dẫn cấu hình và thực thi

5.1. Điều kiện trước khi chạy

- PostgreSQL 16 hoạt động; migration đã được áp dụng.
- Cài dependencies ở thư mục gốc dự án.
- Thiết lập biến môi trường, không commit secret thật.

```
JWT_ACCESS_SECRET=<chuỗi-ngẫu-nhiên-tối-thiểu-32-byte>
JWT_REFRESH_SECRET=<chuỗi-ngẫu-nhiên-khác-tối-thiểu-32-byte>
OTP_HMAC_SECRET=<chuỗi-ngẫu-nhiên-tối-thiểu-32-byte>
GOOGLE_CLIENT_ID=<google-client-id>
GOOGLE_CLIENT_SECRET=<google-client-secret>
GOOGLE_REDIRECT_URI=https://<backend>/api/v1/auth/google/callback
FRONTEND_URL=https://<frontend>
```

Ở môi trường production, callback Google và frontend phải dùng HTTPS. Cookie Refresh Token có thuộc tính Secure, vì vậy trình duyệt chỉ gửi cookie qua HTTPS.

5.2. Chạy backend và kiểm thử

```
cd ~/Desktop/E-healthcare/server
npm test
```

```
npm run build
npm run start:dev
```

5.3. Luồng kiểm tra thủ công

1. Gọi POST `/auth/register/otp` với Email hoặc SĐT và mật khẩu hợp lệ.
2. Lấy OTP từ provider cấu hình cho môi trường thử nghiệm, gọi POST `/auth/register/verify`.
3. Xác nhận User ACTIVE, ROLE_PATIENT và PHR rỗng trong CSDL.
4. Đăng nhập tại POST `/auth/login`; kiểm tra Access Token trong JSON và Refresh Token trong cookie.
5. Gọi GET `/auth/me` với header `Authorization: Bearer <access-token>`.
6. Cấu hình Google credentials, sau đó kiểm tra luồng GET `/auth/google` và callback.

6. Quản lý mã nguồn

Mục	Trạng thái
Nhánh thực hiện	feature/srs-auth-01-registration
Nhánh đích	develop
Commit/Pull Request	Chưa tạo tại thời điểm báo cáo
Phạm vi thay đổi	Auth module, migration, DTO, guard, cấu hình backend và client Auth

Trước khi commit cần rà soát để không đưa secret, .env, dữ liệu thử nghiệm hoặc tệp không thuộc phạm vi vào commit.

7. Hạn chế và công việc tiếp theo

- Cần cấu hình Google credentials và provider Email/SMS thật theo môi trường triển khai để kiểm thử end-to-end.
- Cần chạy migration và integration test trên PostgreSQL của môi trường CI/staging trước khi merge.
- Cần cấu hình HTTPS/Reverse Proxy trước khi triển khai production để cookie Secure hoạt động đúng.
- Cần thêm quan sát vận hành cho số lần OTP thất bại, khóa đăng nhập và lỗi OAuth.

8. Kết luận

Mô-đun xác thực đã đáp ứng toàn bộ 7 hạng mục trong checklist: đăng ký an toàn, OTP có giới hạn, tự tạo PHR, đăng nhập có khóa tài khoản, Google OAuth/OIDC, JWT có Refresh Token an toàn và RBAC bốn vai trò. Backend đã build thành công và 54/54 test

tự động hiện có đã pass. Việc triển khai thực tế còn cần cấu hình secret, Google OAuth, Email/SMS provider và HTTPS theo môi trường.